

HPC3

3. Implement Min, Max, Sum and Average operations using Parallel Reduction.

```
#include <iostream>    // for input-output (cin, cout)
#include <vector>       // for dynamic array (vector)
#include <omp.h>        // for OpenMP parallel programming
using namespace std;

int main() {
    int n;

    // ♦ Take number of elements from user
    cout << "Enter number of elements: ";
    cin >> n;

    // ♦ Create vector of size n
    vector<int> arr(n);

    // ♦ Take array elements as input
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    // ♦ Initialize variables
    int sum = 0;        // to store sum
    int min_val = arr[0]; // initialize min with first element
    int max_val = arr[0]; // initialize max with first element

    // 🔥 Parallel Reduction using OpenMP
    // Each thread calculates partial results and OpenMP combines them
    #pragma omp parallel for reduction(+:sum) reduction(min:min_val) reduction(max:max_val)
    for (int i = 0; i < n; i++) {
```

```

// ♦ Add elements to sum
sum += arr[i];

// ♦ Find minimum value
if (arr[i] < min_val)
    min_val = arr[i];

// ♦ Find maximum value
if (arr[i] > max_val)
    max_val = arr[i];
}

// ♦ Calculate average
double avg = (double)sum / n;

// ♦ Print results
cout << "\nSum = " << sum;
cout << "\nMin = " << min_val;
cout << "\nMax = " << max_val;
cout << "\nAverage = " << avg << endl;

return 0; // end of program
}

/*
g++ -fopenmp -std=c++11 hpc3.cpp -o hpc3
./hpc3
*/

```

Line-by-Line Explanation (Easy Language)

◆ Header Files

```
#include <iostream>
```

👉 Used for input and output (cin, cout)

```
#include <vector>
```

👉 Used to store array dynamically

```
#include <omp.h>
```

👉 Required for OpenMP parallel programming

◆ Namespace

```
using namespace std;
```

👉 Avoid writing std:: again and again

◆ Main Function

```
int main() {
```

👉 Program execution starts here

◆ Input Size

```
int n;
```

```
cout << "Enter number of elements: ";
```

```
cin >> n;
```

👉 Take number of elements from user

◆ Declare Array

```
vector<int> arr(n);
```

👉 Create dynamic array of size n

◆ Input Elements

```
cout << "Enter elements:\n";
for (int i = 0; i < n; i++)
    cin >> arr[i];
```

👉 Store user input values into array

◆ Initialize Variables

```
int sum = 0;
int min_val = arr[0];
int max_val = arr[0];
```

👉 Initialize:

- sum = 0
 - min_val and max_val = first element
-

🔥 🔵 Parallel Reduction (MOST IMPORTANT PART)

```
#pragma omp parallel for reduction(+:sum) reduction(min:min_val) reduction(max:max_val)
```

👉 This line does:

- Divides loop work among threads
 - Each thread works independently
 - Combines results automatically
-

◆ Loop Execution

```
for (int i = 0; i < n; i++) {
```

👉 Loop through all elements

◆ Sum Calculation

```
sum += arr[i];
```

👉 Each thread adds elements to its local sum

◆ Minimum Check

```
if (arr[i] < min_val)
    min_val = arr[i];
```

👉 Update minimum value

◆ Maximum Check

```
if (arr[i] > max_val)
    max_val = arr[i];
```

👉 Update maximum value

◆ Average Calculation

```
double avg = (double)sum / n;
```

👉 Calculate average

◆ Output Results

```
cout << "\nSum = " << sum;
cout << "\nMin = " << min_val;
cout << "\nMax = " << max_val;
cout << "\nAverage = " << avg << endl;
```

👉 Print final results

◆ End Program

```
return 0;
}
```

👉 Program ends

🎯 20-Second Viva Explanation (Use This ¹⁰⁰)

“This program uses OpenMP reduction to compute sum, minimum, and maximum in parallel. Each thread calculates partial results, and OpenMP combines them safely to produce the final result. This avoids race conditions and improves performance.”

⚠ Important Line (Say This for Extra Marks)

“Reduction is more efficient than critical section because it avoids locking overhead.”

Example Input

n = 5

arr = [10, 20, 5, 8, 15]

◆ Step 1: Initialization (Code + State)

```
int sum = 0;
int min_val = arr[0]; // 10
int max_val = arr[0]; // 10
```

👉 Initial values:

```
sum = 0
min = 10
max = 10
```

🔥 Step 2: Parallel Loop Starts

```
#pragma omp parallel for reduction(+:sum) reduction(min:min_val) reduction(max:max_val)
for (int i = 0; i < n; i++) {
```

👉 OpenMP:

- Splits array among threads
 - Each thread gets its **own copy** of sum, min_val, max_val
-

◆ Assume 2 Threads (for understanding)

Thread Elements

T1 [10, 20]

T2 [5, 8, 15]

● Thread 1 Execution (Code Flow)

```
sum += arr[i];
if (arr[i] < min_val) min_val = arr[i];
if (arr[i] > max_val) max_val = arr[i];
```

👉 Step-by-step:

Start:

sum = 0, min = 10, max = 10

i=0 → 10

sum = 10
min = 10
max = 10

i=1 → 20
sum = 30
min = 10
max = 20

👉 Thread 1 result:

sum = 30, min = 10, max = 20

● Thread 2 Execution

Start:

sum = 0, min = 10, max = 10

👉 Step-by-step:

i=2 → 5
sum = 5
min = 5
max = 10

i=3 → 8
sum = 13
min = 5
max = 10

i=4 → 15
sum = 28
min = 5
max = 15

👉 Thread 2 result:

sum = 28, min = 5, max = 15

◆ Step 3: Reduction (Automatic Combine)

👉 OpenMP combines results:

Final Sum = $30 + 28 = 58$

Final Min = $\min(10, 5) = 5$

Final Max = $\max(20, 15) = 20$

◆ Step 4: Average

```
double avg = (double)sum / n;
```

```
avg = 58 / 5 = 11.6
```

✅ Final Output

Sum = 58

Min = 5

Max = 20

Average = 11.6

🔍 Code Flow Summary (Super Important)

1 Initialize variables

2 Parallel loop starts

3 Each thread computes:

- partial sum
- local min
- local max

4 OpenMP combines results

5 Average calculated

6 Output printed

⚙️ Key Line Explanation (VERY IMPORTANT)

reduction(+:sum) reduction(min:min_val) reduction(max:max_val)

👉 Means:

- sum → added across threads
 - min_val → minimum across threads
 - max_val → maximum across threads
-

🎯 Viva Explanation (Say This 100%)

“Each thread processes part of the array and computes partial sum, min, and max. OpenMP reduction combines these partial results safely to produce final output without race condition.”

Compile & Run (Ubuntu)

```
g++ -fopenmp -O2 reduce.cpp -o reduce
./reduce
```

Example Run

Input:

Enter number of elements: 5
Enter elements:
10 20 5 8 15

Output:

Sum = 58
Min = 5
Max = 20
Average = 11.6

What Happened Internally

Array is divided among threads

 Example (2 threads):

Thread Elements

T1 10, 20

T2 5, 8, 15

Each thread computes partial results

 Thread 1:

- Sum = 30
- Min = 10
- Max = 20

 Thread 2:

- Sum = 28
- Min = 5
- Max = 15

OpenMP combines results (Reduction)

👉 Final:

Sum = $30 + 28 = 58$

Min = $\min(10, 5) = 5$

Max = $\max(20, 15) = 20$

Average calculated

$58 / 5 = 11.6$

How to Explain Result in Viva

Say this clearly:

“The array is divided among multiple threads. Each thread calculates partial sum, minimum, and maximum. OpenMP reduction combines these partial results to produce the final output.”

Basic Questions

1. What is reduction in OpenMP?

Answer:

Reduction is a technique where multiple threads compute partial results and OpenMP combines them into a single final result.

2. Why do we use reduction?

Answer:

To avoid **race conditions** when multiple threads update the same variable.

3. What is a race condition?

Answer:

A race condition occurs when multiple threads access and modify shared data simultaneously, leading to incorrect results.

4. What operations support reduction?

Answer:

- Addition (+)
 - Multiplication (*)
 - Minimum (min)
 - Maximum (max)
-

? 5. What does this line do?

```
#pragma omp parallel for reduction(+:sum)
```

Answer:

It divides loop iterations among threads and combines their partial sums into a final value.

 Code-Based Questions

? 6. Why initialize min_val = arr[0]?

Answer:

To start comparison from the first element and ensure correct minimum value.

? 7. Why initialize sum = 0?

Answer:

Because sum starts from zero and accumulates all elements.

? 8. Why do we calculate average separately?

Answer:

Because average = sum / number of elements, so we use the computed sum.

? 9. What happens if reduction is not used?

Answer:

It may cause incorrect results due to race conditions.

? 10. Can we use critical instead of reduction?

Answer:

Yes, but it is slower because it uses locking, while reduction is more efficient.

Conceptual Questions

? 11. How does reduction work internally?

Answer:

Each thread maintains a local copy of the variable and OpenMP combines them at the end.

? 12. Why is reduction faster than critical section?

Answer:

Because it avoids locking and allows parallel computation without synchronization overhead.

? 13. What is parallelism?

Answer:

Parallelism is executing multiple tasks simultaneously using multiple cores.

? 14. What is OpenMP?

Answer:

OpenMP is an API used for parallel programming in C/C++ on shared memory systems.

? 15. What is a thread?

Answer:

A thread is a lightweight unit of execution within a process.

Important Practical Questions

? 16. When does parallel reduction give better performance?

Answer:

When input size is large and workload is enough to utilize multiple threads.

? 17. Why parallel may be slower sometimes?

Answer:

Due to thread creation overhead and synchronization cost.

? 18. What is scalability?

Answer:

Ability of a program to improve performance when more processors are added.

? 19. What is speedup?

Answer:

$\text{Speedup} = \text{Sequential Time} / \text{Parallel Time}$

? 20. What is the limitation of this program?

Answer:

For small input sizes, parallel overhead may reduce performance benefits.

 **Final Killer Answer (Use This )**

If examiner asks:

 *"What is your conclusion?"*

Answer:

"Parallel reduction efficiently computes aggregate operations like sum, minimum, and maximum by dividing work among threads and combining results automatically. It avoids race conditions and reduces execution time for large datasets."

 **Bonus Smart Line (Topper Level)**

"Reduction provides better performance than critical section because it minimizes synchronization overhead."